API-Only LLM Baseline for GNN-CB

Adapting the ineqmath Evaluation Process

Murad Hossen

GNN RISING STAR

May 13, 2026

Slide 1 — ineqmath Process vs Our GNN-CB Adaptation

ineqmath (Original Paper)

1. Frozen Prompt (math problem)
2. API Call (Together AI / Anthropic)
3. LLM returns text answer
4. Parse answer (no execution)
5. Compute Score (exact match)

GNN-CB (Our Adaptation)

1. Frozen Prompt (GNN task)
2. API Call (Together AI)
3. LLM returns Python code
4. **Execute code (subprocess) [NEW]**
5. **Validate submission.csv [NEW]**
6. **If fail: repair loop k$\leq$5 [NEW]**

Borrowed the **API call pattern** from ineqmath and added **code execution + repair loop** — GNN-CB needs runnable code, not text answers.

Slide 2 — Why Hugging Face + Together AI? (Models 1–9)

Hugging Face
Model Weights
(Storage / Source)

downloads weights →

Together AI
GPU Cloud
(Inference / API)

REST API endpoint →

Our Code
llama_api_mode.py
(API call)

text response →

Paper cites this
(reference only)

Paper calls this
(actual inference)

`Together(api_key)`
+ `chat.completions`

Why Hugging Face?

- **Universal hub** — every AI lab publishes there
- GitHub of AI models
- Paper links HF to **cite** the model (like a DOI)
- HF is never called at runtime

Why Together AI?

- Hosts ALL open-source models on GPU cloud
- **One API** for Llama, Qwen, Mistral, DeepSeek
- Same format as OpenAI API
- Free serverless tier available

Slide 3 — The API Engine: ineqmath vs Our Implementation

ineqmath — engines/together.py

```python
class ChatTogether(EngineLM, CachedEngine):
  # inherits TextGrad + caching
  def generate(self, content, ...):
    response = self.client\
      .chat.completions.create(
        model=self.model_string,
        messages=messages,
        temperature=temperature,
        top_p=top_p,
        max_tokens=max_tokens,
    )
    return response.choices[0]\
               .message.content
```

Run via shell script:

```bash
# run_dev_data_together.sh
for LLM in "${ENGINES[@]}"; do
  python solve.py --llm_engine_name $LLM
done
```

Ours — llama_api_mode.py

```python
# Plain function      same API call
def call_llm(client, messages):
  response = client\
    .chat.completions.create(
      model=MODEL,
      messages=messages,
      temperature=TEMPERATURE,  # 0
      top_p=TOP_P,              # 0.99
      max_tokens=MAX_OUTPUT_TOKENS,
  )
  return response.choices[0]\
             .message.content
```

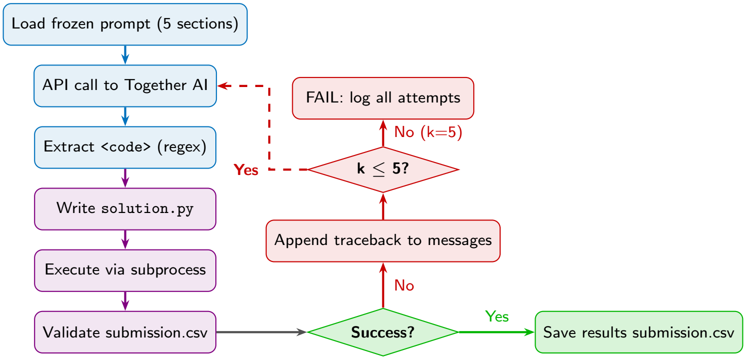
Run via single command:

```bash
# No shell script (one model only)
export TOGETHER_API_KEY="..."
python llama_api_mode.py
```

Key difference

ineqmath wraps the call in a TextGrad class with caching for batch processing. We use a plain function — **same underlying API call**, no extra dependencies.

Slide 4 — The Repair Loop: Core Innovation for GNN-CB



ineqmath — No repair loop

LLM returns text answer → parse → done. No code runs.

GNN-CB — Repair loop essential

LLM produces runnable code. Errors fed back. LLM retries up to k=5.

Slide 5 — Results & Output Structure

Run Configuration

Mode	API-only (non-agent)
Provider	Together AI (free tier)
Model	Llama-3.3-70B-Turbo
Temp.	0 (greedy)
Top-p	0.99
Repairs	3 / 5 used
Time	74 seconds

Performance Comparison

Mode	**Macro-F1**	**Repairs**
Agent (Claude Opus 4.7)	-	N/A
API-only (Llama-3.3-70B)	0.46717	3/5
Human Best	0.537	—

Output folder: `run_llama_api/`

- `config.json` — hyperparameters
- `environment.txt` — pip freeze
- `prompt_filled.md` — exact prompt
- `attempt_{k}_raw_response.txt`
- `attempt_{k}_solution.py`
- `attempt_{k}_execution_log.txt`
- `final_solution.py` — winning code
- `submission.csv` — 36 predictions
- `llm_llama33_70b.csv.enc` — encrypted
- `run_summary.csv` — summary